ChatGPT for developers: practical guide

Prompt engineering foundations

Getting ChatGPT to write good code is basically like explaining what you need to a junior dev. You gotta be specific, give context, and tell it exactly what format you want. Once you nail this formula, you'll get way better outputs every single time.

The power prompt formula

```
Role + Context + Task + Constraints + Format = Perfect Output
```

Sample prompt:

```
"Act as a senior React developer with Next.js 15 expertise. I'm building an e-c
ommerce site and getting hydration mismatches when cart data updates on th
e client side. The cart uses Redux for state management. Debug this issue an
d provide a solution that keeps SSR working properly. Don't suggest removing
Redux or doing a major refactor. Give me step-by-step implementation with ac
tual code examples I can copy paste."
```

Essential prompt modifiers

These little phrases are like cheat codes. Just tack them onto any prompt and watch the quality jump. I use them constantly.

```
"Include TypeScript types"
"Follow React best practices"
"Add error handling for edge cases"
"Make it accessible (WCAG 2.1)"
"Optimize for mobile first"
"Use modern ES6+ syntax"
"Add JSDoc comments for complex logic"
```

"Include at least one usage example"
"Make sure it works with React strict mode"

Code generation

You can literally describe a component like you're talking to a coworker and it'll generate working code. The trick is being super detailed about features and edge cases.

React component generator

"Build me a SearchableMultiSelect component in TypeScript. It needs to handl e 10,000+ options without lagging, so use virtualization. Users should be able to select multiple items, search through them, and remove selections with a litt le X button. Make it fully keyboard accessible with arrow keys and enter. Style it with Tailwind but make the styles customizable through className props. O h and add loading and error states. The search should debounce at 300ms. W hen nothing matches the search, show a friendly empty state message."

Next.js API route builder

"Create a Next.js 154 API route using the app directory at /api/users/[userId]/ posts. It should handle GET to fetch user posts with pagination (cursor-based, 20 per page), POST to create a new post with Zod validation for title (required, max 200 chars) and content (required, max 5000 chars). Add rate limiting at 3 0 requests per minute per IP. Include proper TypeScript types for request and response. Return appropriate status codes and error messages. Also add COR S headers for my frontend at localhost:3000."

Custom hook creator

You know those hooks you write over and over? Just describe the behavior you want and boom, reusable hook ready to go.

"Write a useLocalStorage hook in TypeScript that syncs state with localStorage. It should work exactly like useState but persist the value. Handle SSR properly (no errors in Next.js). Include JSON serialization for objects. If localStorage is unavailable or throws an error, fall back to regular state. Add an option to sync across browser tabs using the storage event. The hook should be generic and type-safe. Include a clear() method to remove the item from storage."

State management setup

State management doesn't have to be a pain. Just describe your data flow and let ChatGPT handle the boilerplate.

"Set up a Zustand store for a shopping cart that handles: items array with product id, name, price, quantity, and image. Include actions for addItem (increment quantity if exists), removeItem, updateQuantity, clearCart, and applyDiscountCode. Calculate computed values for subtotal, tax (8.5%), shipping (free over $50, otherwise $10), and total. Persist the cart to localStorage but encrypt sensitive data. Add TypeScript types for everything. Include middleware to log all actions in development mode only."

Debugging & error fixing

We've all been there. 2am, error makes no sense, Stack Overflow isn't helping. This is when ChatGPT becomes your debugging buddy. Just dump everything you know about the problem and let it work through it.

Debug framework

"Help me debug this React issue. My useEffect is causing infinite re-renders:

```
useEffect(() => {
  const filtered = data.filter(item => item.category === category);
  setFilteredData(filtered);
}, [data, category, filteredData]);
```

The component keeps re-rendering infinitely when I change the category. I'm using React 18 with TypeScript in a Vite project. The data comes from React Query and category is from URL params. I need to filter the data whenever category changes but this is freezing the browser. What's causing this and how do I fix it properly?"

Performance optimization

Performance issues are tricky but ChatGPT is great at spotting the obvious stuff we miss when we're too close to the code.

"This product listing component is super laggy with 500+ products:

```
const ProductList = ({ products }) => {
  const sortedProducts = products.sort((a, b) => b.rating - a.rating);

  return sortedProducts.map(product => (
    <ProductCard
      key={product.id}
      product={product}
      onClick={() => trackEvent('product_click', product)}
    />
  ));
}
```

Profiler shows each render takes 400ms. Target is under 16ms for smooth 60fps. The ProductCard is already wrapped in React.memo. What optimizations would have the biggest impact? Rank them by how much they'll actually help."

Memory leak detection

Memory leaks are the worst. Your app works fine for 5 minutes then turns into a slideshow. Here's how to track them down.

"Find memory leaks in this component that subscribes to WebSocket events:

```
const ChatRoom = ({ roomId }) ⇒ {
  const [messages, setMessages] = useState([]);

  useEffect(() ⇒ {
    const ws = new WebSocket(`wss://api.example.com/rooms/${roomId}`);

    ws.onmessage = (event) ⇒ {
      setMessages(prev ⇒ [...prev, JSON.parse(event.data)]);
    };

    window.addEventListener('beforeunload', () ⇒ {
      ws.send(JSON.stringify({ type: 'LEAVE_ROOM' }));
    });
  }, [roomId]);

  return <MessageList messages={messages} />;
}
```

Chrome DevTools shows memory growing from 50MB to 500MB after 10 minu tes. Heap snapshots show detached DOM nodes and event listeners. What's l eaking and how do I fix it?"

Code review & refactoring

Refactoring is like cleaning your room. You know you should do it, but it's hard to know where to start. These prompts help you systematically improve your code without breaking things.

Clean code refactor

"Refactor this messy React component to follow clean code principles:

```
const UserDashboard = () ⇒ {
  const [d, setD] = useState(null);
  const [l, setL] = useState(true);
```

```
  const [e, setE] = useState(null);

  useEffect(() => {
    fetch('/api/user').then(r => r.json()).then(data => {
      setD(data);
      setL(false);
    }).catch(err => {
      setE(err.message);
      setL(false);
    });
  }, []);

  if(l) return <div>Loading...</div>;
  if(e) return <div>{e}</div>;

  return <div>
    <h1>{d?.n}</h1>
    <p>{d?.e}</p>
    <button onClick={() => fetch('/api/logout').then(() => window.location.href =
'/')}>
      Logout
    </button>
  </div>;
}
```

Fix: terrible variable names, no error handling, inline functions, no types, mixe
d concerns. Make it readable and maintainable."

Design pattern application

Sometimes your code works but the structure is all wrong. Design patterns fix that.

"Apply the Compound Component pattern to this Tab component:

const Tabs = ({ tabs, activeTab, onTabChange }) => {

```
  return (
    <div>
      <div className='tab-list'>
        {tabs.map(tab => (
          <button
            key={tab.id}
            className={activeTab === tab.id ? 'active' : ''}
            onClick={() => onTabChange(tab.id)}
          >
            {tab.label}
          </button>
        ))}
      </div>
      <div className='tab-content'>
        {tabs.find(t => t.id === activeTab)?.content}
      </div>
    </div>
  );
};
```

Refactor this to use Compound Components pattern like Radix UI or Headless UI. Should be composable and flexible. Include TypeScript types and usage e xample."

Accessibility audit

Accessibility isn't optional anymore. But it's easy to miss stuff if you don't know what to look for.

```
"Review this form component for accessibility issues:

const ContactForm = () => {
  const [error, setError] = useState('');

  return (
    <div className='form'>
```

```
      <input type='text' placeholder='Your name' />
      <input type='email' placeholder='Email' />
      <textarea placeholder='Message'></textarea>
      {error && <span style={{color: 'red'}}>{error}</span>}
      <div className='submit-btn' onClick={handleSubmit}>
        Send Message
      </div>
    </div>
  );
}
```

Check for WCAG 2.1 AA compliance. Look for keyboard navigation, screen reader issues, color contrast, focus management, and semantic HTML problems. List all issues with severity levels and fixes."

Testing automation

Writing tests is tedious but ChatGPT makes it almost fun. Well, maybe not fun, but definitely faster. Just show it your code and tell it what testing library you use.

Unit test generator

"Write comprehensive Jest + React Testing Library tests for this custom hook:

```
const useDebounce = (value, delay = 500) => {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);
```

```
  return debouncedValue;
};

Test: value updates after delay, cleanup on unmount, immediate update when delay is 0, handles rapid value changes, works with different data types. Use renderHook from @testing-library/react-hooks. Include edge cases I might not have thought of."
```

E2E test creator

E2E tests are crucial but annoying to write. This prompt handles the boring parts.

```
"Create Playwright E2E tests for this checkout flow:
1. User adds item to cart from product page
2. Opens cart drawer
3. Updates quantity
4. Applies coupon code 'SAVE20'
5. Proceeds to checkout
6. Fills shipping info
7. Enters credit card
8. Places order

Test the happy path plus these scenarios: invalid coupon, out of stock item, payment failure, network timeout during submission. Use Page Object Model pattern. Include mobile viewport tests. Add screenshots on failure. Make tests resilient with proper waits and retries."
```

API integration tests

Integration tests catch those weird bugs that only happen when everything's hooked up together.

```
"Write Vitest integration tests for this tRPC router:

export const userRouter = router({
  getById: publicProcedure
```

```
    .input(z.object({ id: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return await ctx.db.user.findUnique({ where: { id: input.id } });
    }),

  updateProfile: protectedProcedure
    .input(z.object({
      name: z.string().min(1).max(100),
      bio: z.string().max(500).optional()
    }))
    .mutation(async ({ input, ctx }) => {
      return await ctx.db.user.update({
        where: { id: ctx.session.userId },
        data: input
      });
    })
});
```

Test: successful queries, validation errors, auth requirements, database error
s, concurrent updates. Mock Prisma client and session. Check response shap
es match types."

Documentation

Good docs are the difference between a library people love and one they rage quit. But writing them is boring. Let ChatGPT handle the tedious parts while you focus on the important stuff.

Component documentation

```
"Document this React component for our component library:

interface DataTableProps<T> {
  data: T[];
  columns: Column<T>[];
```

```
  onRowClick?: (row: T) => void;
  loading?: boolean;
  sortable?: boolean;
  pagination?: PaginationConfig;
  emptyMessage?: string;
}

const DataTable = <T,>({ data, columns, ...props }: DataTableProps<T>) => {
  // implementation
}
```

Write docs including: component overview explaining when to use it vs regula r tables, complete props table with types and defaults, 5 different usage exam ples (basic, sortable, paginated, clickable rows, custom cells), common pitfall s, performance tips for large datasets, migration guide from v1, accessibility f eatures. Format in MDX for Storybook."

API documentation

Nobody likes writing API docs but everybody hates when they're missing. This makes it painless.

```
"Generate OpenAPI documentation for these Next.js API routes:

// GET /api/products?category=electronics&minPrice=0&maxPrice=1000&pag
e=1&limit=20
// Returns paginated products with filters

// POST /api/products
// Creates new product (admin only)
// Body: { name, description, price, category, stock, images[] }

// PATCH /api/products/[id]
// Updates product fields (admin only)

// DELETE /api/products/[id]
```

```
// Soft deletes product (admin only)

Include: detailed descriptions, request/response schemas with examples, all s
tatus codes with meanings, authentication requirements, rate limiting (100/hou
r for GET, 10/hour for mutations), example requests in cURL and JavaScript fet
ch. Format as OpenAPI 3.0 YAML."
```

Performance & optimization

Performance is one of those things that's easy to ignore until your users start complaining. These prompts help you find and fix bottlenecks before they become real problems.

Bundle size optimization

Nothing kills performance like shipping 5MB of JavaScript. Here's how to slim down.

```
"Our Next.js app's initial bundle is 2.3MB and it's killing our Core Web Vitals.
Main bundle includes: React, NextUI components, Framer Motion, date-fns, lo
dash, axios, three different icon libraries.

Analyze and suggest: which packages have smaller alternatives (with migratio
n effort), what should be dynamically imported, what can be loaded from CD
N, tree shaking opportunities, webpack config optimizations. Goal is under 50
0KB initial bundle. Rank suggestions by impact vs effort."
```

React render optimization

When your React app feels sluggish, it's usually because something's re-rendering way too much.

```
"Optimize this component that's re-rendering 50+ times per second:

const Dashboard = () => {
  const [searchTerm, setSearchTerm] = useState('');
```

```
  const [data, setData] = useState([]);

  const filteredData = data.filter(item =>
    item.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  const stats = {
    total: filteredData.length,
    active: filteredData.filter(item => item.active).length,
    averageValue: filteredData.reduce((sum, item) => sum + item.value, 0) / filte
redData.length
  };

  return (
    <>
      <SearchInput onChange={setSearchTerm} />
      <StatsCards stats={stats} />
      <DataGrid data={filteredData} />
    </>
  );
}
```

React DevTools shows everything re-renders on every keystroke. Fix with useMemo, useCallback, and React.memo. Show me exactly where to apply each optimization and explain why."

Core web vitals improvement

Google cares about these metrics, so you should too. Bad scores mean worse SEO.

"Fix Core Web Vitals for our Next.js e-commerce site:

Current scores:
- LCP: 4.5s (hero image loads slowly)
- FID: 180ms (JavaScript blocking main thread)

- CLS: 0.31 (layout shifts when fonts and images load)

The site has: hero carousel with 5 images, custom fonts, 50+ product cards wi th images, third-party scripts (Google Analytics, Intercom, Stripe)

Provide specific fixes for each metric with implementation code. Include next/i mage config, font loading strategy, and script optimization. Need all metrics in green (LCP < 2.5s, FID < 100ms, CLS < 0.1)."

Architecture & system design

System design is where you go from "it works on my machine" to "it works for millions of users". These prompts help you think through the big picture stuff.

Microservices architecture

Breaking up a monolith is scary but sometimes necessary. This helps you plan it out.

"Design microservices architecture for our e-commerce platform. Current mo nolith Next.js app handles everything: user auth, product catalog, inventory, or ders, payments, emails, admin dashboard.

We're hitting scaling issues at 50K daily users. Database is getting huge (500 GB PostgreSQL). Different teams want to deploy independently.

Split into logical services with: clear boundaries, API design between service s, database strategy (shared vs separate), event bus for communication, auth entication flow, deployment approach. Keep it practical, not over-engineered. We have 5 developers total."

Database schema design

Good schema design early on saves so much pain later. Get it right from the start.

"Design PostgreSQL schema for a Discord-like app using Prisma:

Features needed:
- Users with profiles and online status
- Servers with channels (text and voice)
- Messages with replies and reactions
- Direct messages between users
- Roles and permissions per server
- Message attachments and embeds
- Read receipts and typing indicators

Requirements: optimize for message queries (newest first with pagination), ha
ndle 1M+ messages per channel, support full-text message search, design for horizontal scaling later. Include indexes, constraints, and explain relationship c
hoices."

Quick productivity hacks

These are the little prompts I use 20 times a day. Simple stuff that saves a few minutes each time but adds up to hours.

Type generator from JSON

Every time an API returns JSON, you need types. This is the fastest way.

"Generate TypeScript types from this API response:

```
{
  "user": {
    "id": "usr_abc123",
    "email": "john@example.com",
    "profile": {
      "firstName": "John",
      "lastName": "Doe",
      "avatar": "https://...",
```

```
      "preferences": {
        "theme": "dark",
        "notifications": true
      }
    },
    "subscription": {
      "plan": "pro",
      "status": "active",
      "expiresAt": "2024-12-31T00:00:00Z"
    }
  },
  "posts": [
    {
      "id": "post_xyz",
      "title": "My Post",
      "views": 1523,
      "published": true,
      "tags": ["react", "typescript"]
    }
  ]
}
```

Create interfaces with proper naming, make optional fields that might be null, use enums for status/plan fields, add utility types for common operations, gen erate Zod schema for runtime validation too."

CSS to Tailwind converter

Converting old CSS to Tailwind is mind-numbing. Let ChatGPT do it.

```
"Convert this CSS to Tailwind classes:

.card {
  background: white;
  border-radius: 12px;
  padding: 24px;
```

```
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  transition: transform 0.2s ease;
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 12px rgba(0, 0, 0, 0.15);
}

@media (max-width: 768px) {
  .card {
    padding: 16px;
    border-radius: 8px;
  }
}
```

Include dark mode variants, show me how to extract this as a reusable compo
nent class if it's used multiple times."

Error message decoder

We've all gotten that cryptic error that makes no sense. This helps decode it.

"Explain this Next.js error in plain English:

Error: Hydration failed because the initial UI does not match what was rendere
d on the server.
Warning: Expected server HTML to contain a matching in .
    at div
    at CartProvider
    at Layout

This happens when I refresh the page but not during navigation. The CartProv
ider uses localStorage to persist cart data. Tell me exactly what's wrong and h

ow to fix it step by step. Don't just say 'use useEffect', show me the actual co de changes needed."

Power user strategies

Once you get comfortable with basic prompts, these strategies take you to the next level. It's like unlocking ChatGPT's hidden features.

Multi-step debugging workflow

Instead of dumping everything in one massive prompt, break it down into steps. Way more effective.

Step 1: "What are the 5 most likely causes for 'Cannot read property of undefi ned' in a React component?"

Step 2: "I think it's async data loading. How do I verify this using React DevTo ols?"

Step 3: "Confirmed it's async data. Write a custom hook that handles loading states properly"

Step 4: "Add TypeScript types to that hook to prevent this error in the future"

Step 5: "Write a unit test that would have caught this bug"

Context preservation template

Starting every prompt with the same context is annoying. Set it once and reference it.

"For all my following prompts, remember this project context:
- Next.js 14 with App Router
- TypeScript with strict mode
- Tailwind CSS for styling
- Prisma with PostgreSQL

```
- tRPC for API
- React Query for data fetching
- Deployed on Vercel
- Target audience: mobile users in Southeast Asia (optimize for slow 3G)
- Business: B2B SaaS for inventory management

Use these conventions in all code you generate."
```

Progressive enhancement chain

Building features incrementally catches issues early and makes debugging easier.

```
Prompt 1: "Create the HTML structure for an image carousel"
Prompt 2: "Add CSS to make it responsive and good looking"
Prompt 3: "Add JavaScript for next/previous functionality"
Prompt 4: "Convert to a React component with TypeScript"
Prompt 5: "Add keyboard navigation and touch gestures"
Prompt 6: "Optimize for performance with lazy loading"
Prompt 7: "Add tests covering all interactions"
```

Remember

ChatGPT is amazing but it's not perfect. Always review the code, especially for production. Test everything, particularly edge cases it might have missed. Check that packages are the latest versions. Run a security scan on anything dealing with user data. Make sure it actually follows your team's style guide. And please, understand what the code does before you ship it.

Pro tip: Keep a `prompts.md` file in your repo with prompts that worked well for your specific codebase. Your future self will thank you.